

COMP1521

Wk3 Tutorial

Directives: Summary

Important Directives - Summary

`.align N`

Adds automatically-recalculated at runtime buffer room until the next address which is a factor of 2^N . Not used often, usually unnecessary!

`.asciiz "string"`

Makes a null-terminated string.

`.space n`

Allocates *n* bytes of space immediately following whatever came before. The space is initially uninitialised! Not used often.

`.byte [val, ...]`

Makes a **one byte** (or series of one byte) slot in memory storing the value.

`.half [val, ...]`

Makes a **two byte** (or series of two byte) slot in memory storing the value. Starts the slot at the next address divisible by 2.

`.word [val, ...]`

Makes a **four byte** (or series of four byte) slot in memory storing the value. Starts the slot at the next address divisible by 4.

Translating MIPS: Arrays

A Note on Loading/Storing Syntax

- The structure of the loading/storing instructions is different to other instructions, which is a pretty common source of errors when it comes to storing in particular!

Most instructions (including la and li):

<code>add</code>	<code>\$t0,</code>	<code>\$t1, \$t2</code>
instruction	register where instruction	other registers/consts/labels
keyword	output is stored	necessary (instruction specific)

Load/store instructions:

<code>lw</code>	<code>\$t0,</code>	<code>(\$t1)</code>
instruction	register whose contents we	label of the storage location or register
keyword	are loading in or storing	holding address of the storage location

Tips To Approach Array Loading/Storing

1. If storing, make sure you have the to be stored value already in a register!
2. Calculate the needed index first.
3. Multiply by the size of an array element in bytes
 - e.g multiply by 4 for indexing into an int array
4. Choose whatever address-specifying syntax you find easiest to use!

(I prefer the *array_label(offset in register)* syntax!)

```
scan_loop:
scan_loop_init:
    li    $t0, 0                # int i = 0;

scan_loop_cond:
    bge   $t0, N_SIZE, scan_loop_end    # while (i < N_SIZE)

scan_loop_body:
    # Step 1 - get what we're storing in a register ($t1)
    li    $v0, READ_INT
    syscall
    move   $t1, $v0

    # Step 2 - calculate the index.
    move   $t2, $t0

    # Step 3 - convert index to offset in bytes
    mul    $t2, $t2, SIZE_OF_INT

    # Step 4 - Choose preferred address-specifying syntax.
    sw     $t1, numbers($t2)

scan_loop_step:
    addi   $t0, 1                # i++;
    j      scan_loop_cond

scan_loop_end:
```

Pointer Arithmetic and MIPS

Pointer Arithmetic In C (1/2)

- The number of bytes that a pointer moves by when executing "some_ptr++" depends on the type!
- In general - "some_ptr++" moves the pointer forward *by the size of the pointer type*!

```
int_ptr++;  
char_ptr++;
```

Instructions performed

```
Initial int pointer stored value: 0x502000000030  
Initial char pointer stored value: 0x502000000050  
  
Post-increment int pointer stored value: 0x502000000034  
Post-increment char pointer stored value: 0x502000000051  
  
Number of bytes moved along in int_ptr++ step: 4  
Number of bytes moved along in char_ptr++ step: 1
```

Under-the-hood changes in pointer position

Pointer Arithmetic In C (2/2)

- Be careful when you translate pointer arithmetic into MIPS – you must account for this manually!

```
int_ptr++;  
char_ptr++;
```

Instructions performed

```
SIZE_OF_INT = 4  
SIZE_OF_CHAR = 1  
  
main:  
    # $t0 corresponds to int_ptr,  
    # $t1 corresponds to char_ptr  
  
    # When need to update (e.g step  
    # through a loop):  
    addi    $t0, $t0, SIZE_OF_INT  
    addi    $t1, $t1, SIZE_OF_CHAR
```

Corresponding MIPS